

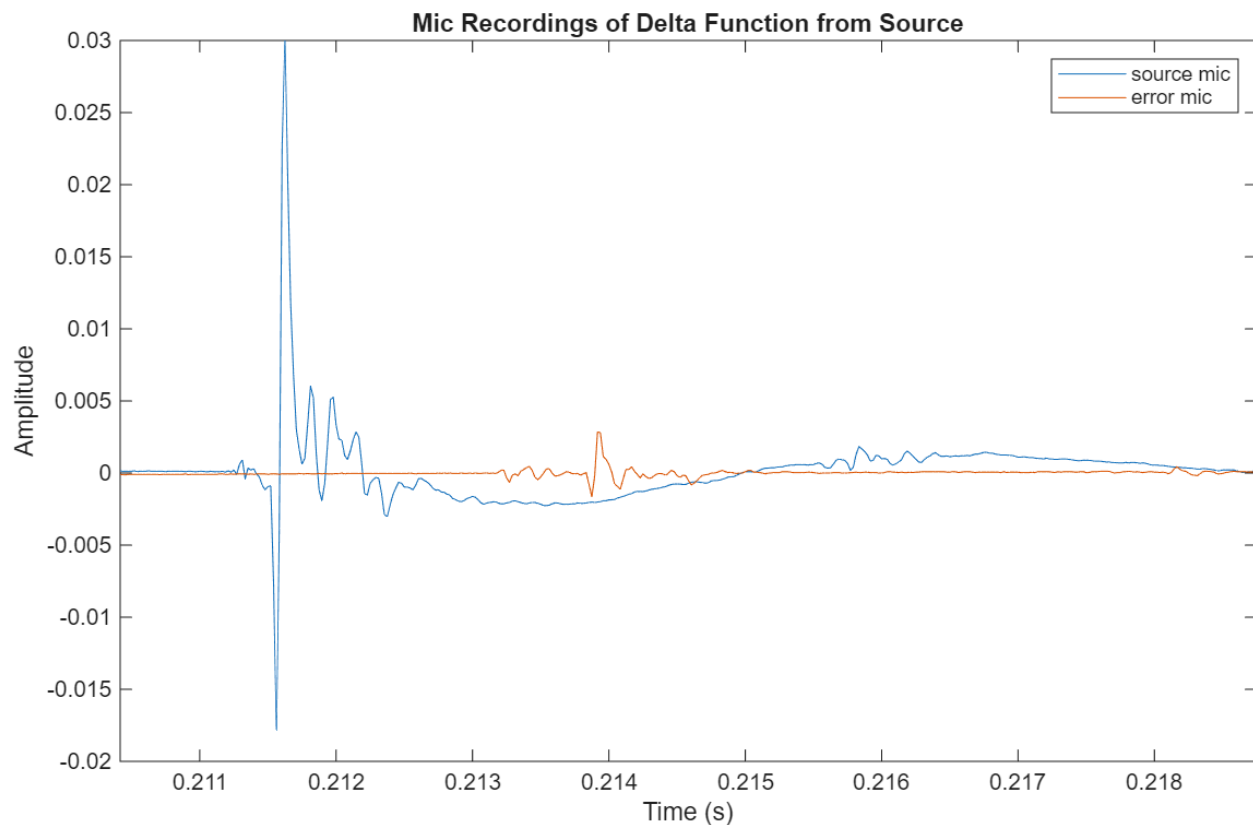
April-29-26

Next steps:

- Try offline least squares cancellation
- Compare offline method filter to learned adaptive filter
- Room EQ Wizard?
- ARTA?
- Manually run a *chirp()* through Matlab?

It is as we feared, the Matlab IR measurer is not consistent at all. There are just random shifts in time, and slight differences in peaks sometimes. There's a latency compensation method, and it seems to plot everything correctly at the start in the window of the tool, but the exported data is not shifted. And it gives us a recorded latency in samples, but it was too many samples, and said it should be shifted to start after the peak occurred. It was almost 1k samples off. Same issue with the MLS method.

I tried playing a click through the source and plotting the 2 mic recordings. We saw pretty much exactly what was expected, although it seems like there's a lot of delay either through focusrite or simulink. The click is at .1s in the source, so it's strange that the mic picks up at .2115s. I want to



April-27-26

Next steps:

- Check Matlab's IR tool
- Try playing a click through the source and look at the mic recordings

Today Dr. Heilemann came into the lab with me to look at the setup and try to eliminate any potential problems. He found me another mic stand instead of using the table stand, we switched panels, and we clipped the panel to the table stand instead of resting it on foam. We also covered the floor with foam just to ensure no extra reflections were being recorded.

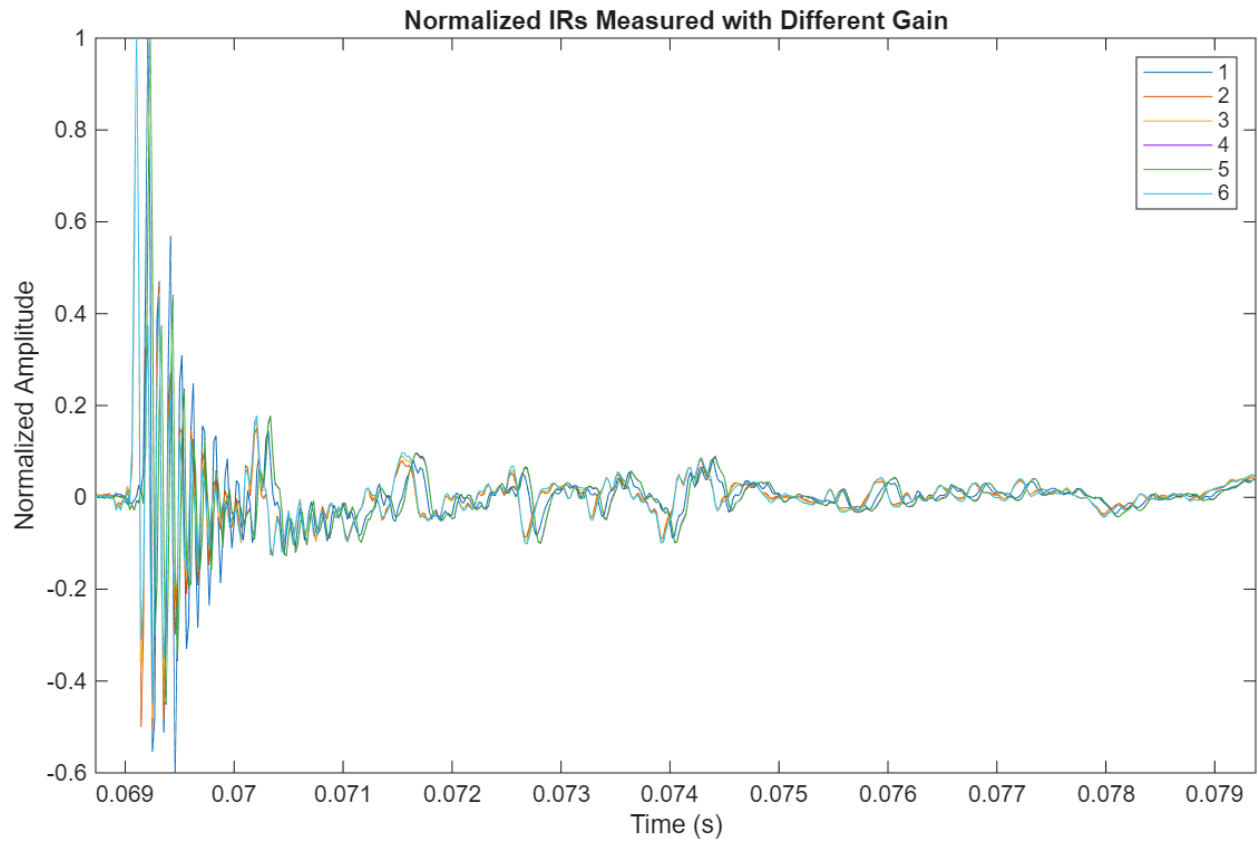
When we tried using the IR measurer, the big panel I was using sounded unstable, even at the mid volume range. We started the sine sweep at 150 Hz instead of 80 Hz, and that helped, but didn't fix it. We switched to the first panel we used at the start of the year, and that one is more stable, although it will have less reflections. We might need to put a board behind it to create more reflections for the panel to cancel.

Now that we have a stable panel, we need to ensure the room isn't causing problems for us. I'm gonna first test the Matlab IR tool to make sure it can record IRs consistently, and then I'm gonna play a delta function from the source to ensure the error mic picks up a tap a little bit delayed after it bounces off the panel. If we don't see that, then something is definitely wrong with the room and we need to fix that before proceeding.

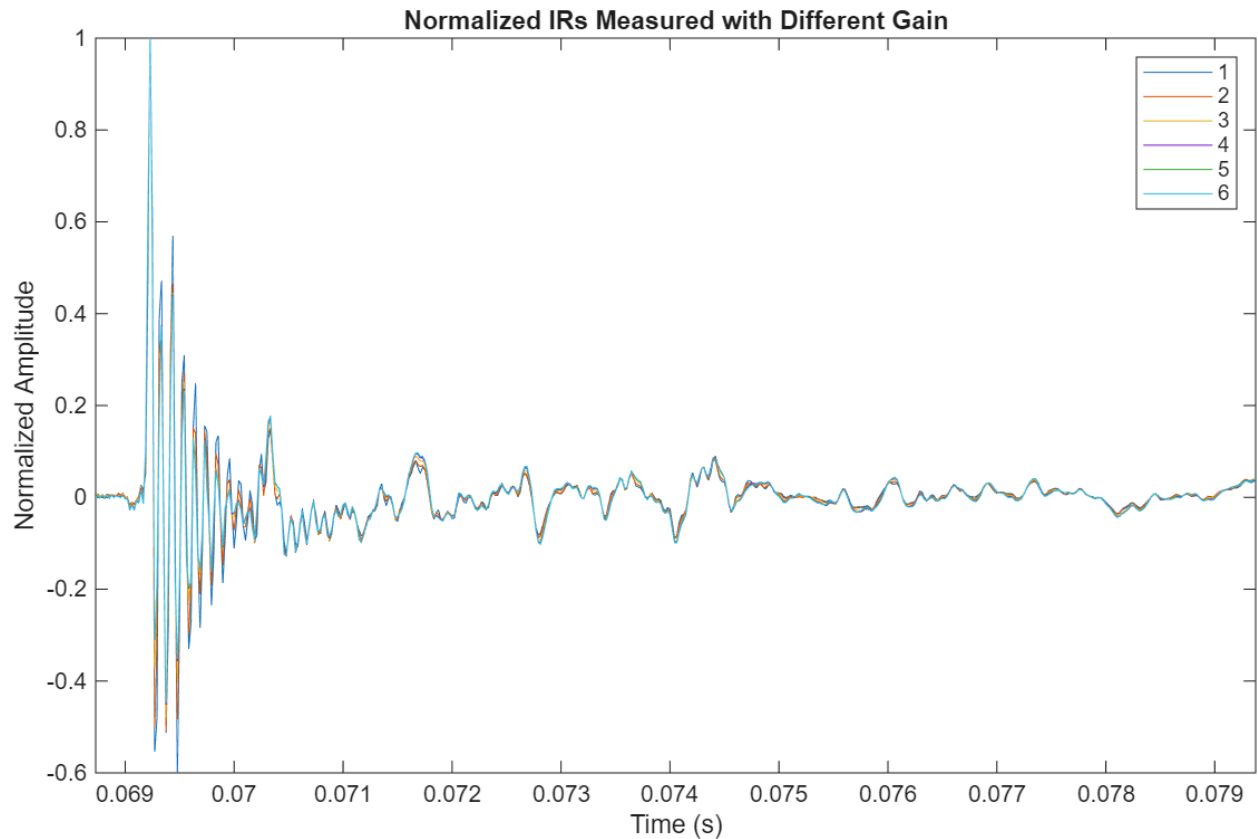
April-21-26**Next steps:**

- Check Matlab's IR tool
- Try cancellation with the dial at like 11:00-12:00

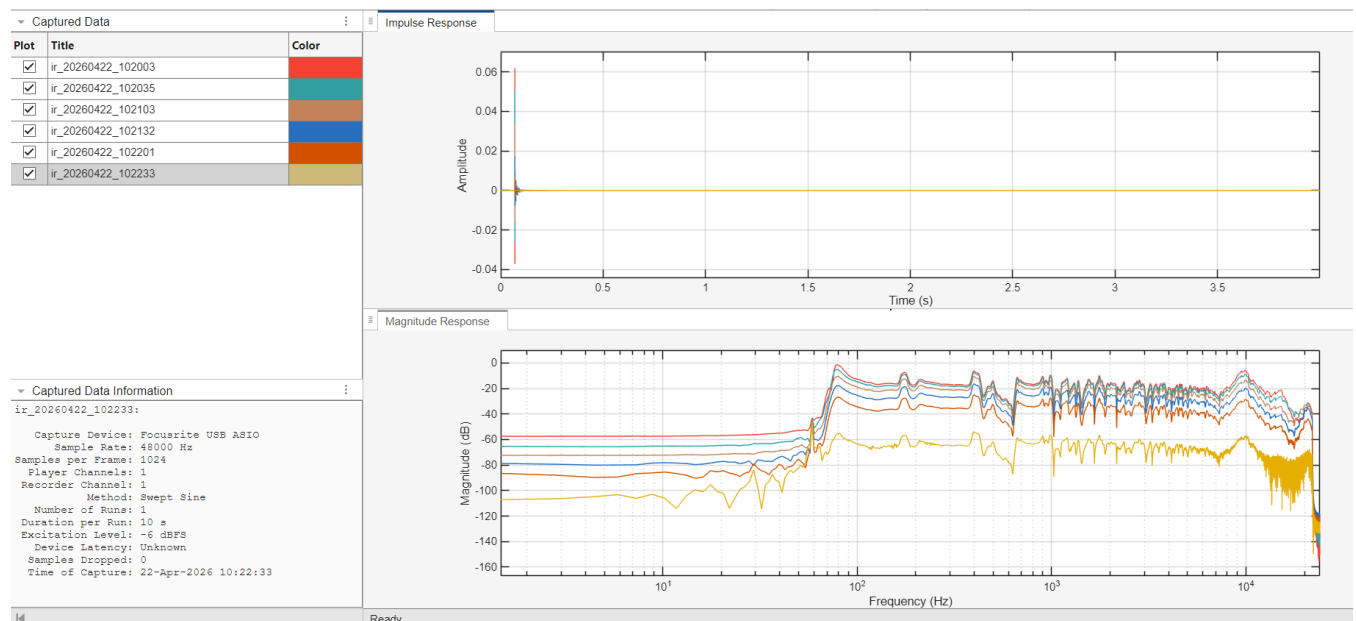
My strategy to test if the exciters are linear is by measuring the IRs from the panel to the error mic with different gains on the focusrite, then normalizing to $[-1, 1]$ and comparing. I ended up frying like 3 drivers trying this, the gain can't be past 3:00 with the big amp or it risks frying. I was able to get some measurements lower than that though.

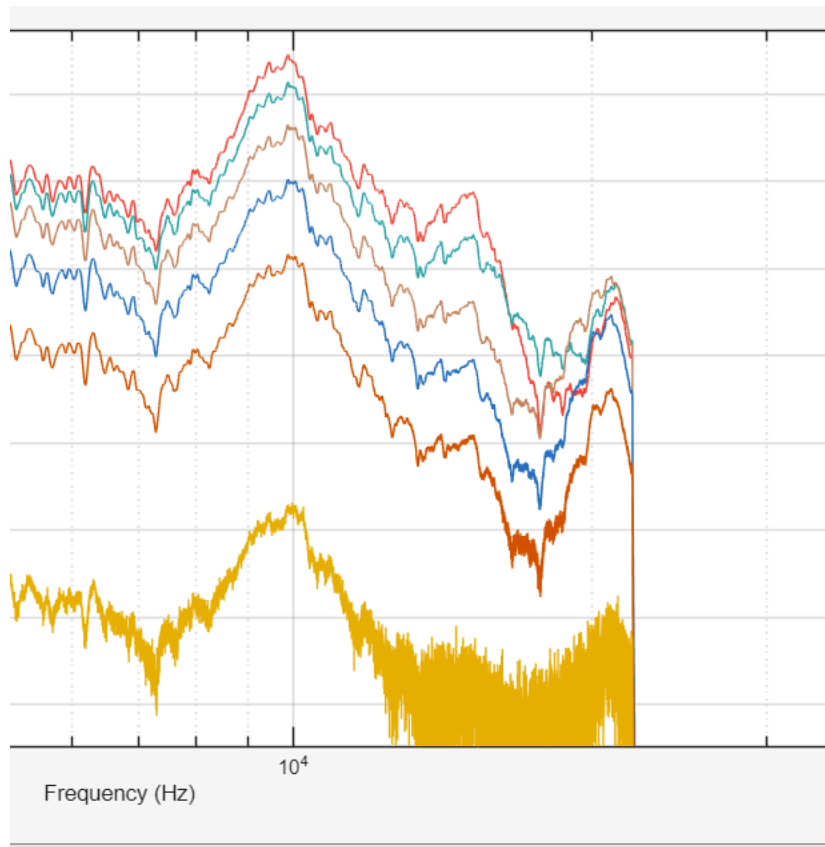


The numbers are in increasing order from highest gain to lowest gain, i.e. 1 was highest gain, 6 was lowest gain. Clearly they aren't all the same. They look kind of similar, but there are some delays and maybe some distortions. 2, 3, and 6 are time aligned, 4 and 5 are time aligned, and 1 is on its own. If we align them all, we get this plot



The highest gain has probably the most distortion, and the lowest gain as well implying that the sweet spot is closer to the middle. When time aligned, 3, 4, and 5 are very similar. I think the ideal range is probably around 12:00 on the gain dial. But it's not clear why it needs to be time aligned at all. I need to check to see if matlabs IR measurer actually does its job correctly. I'm gonna try taking the same IR multiple times and see if it's the same.



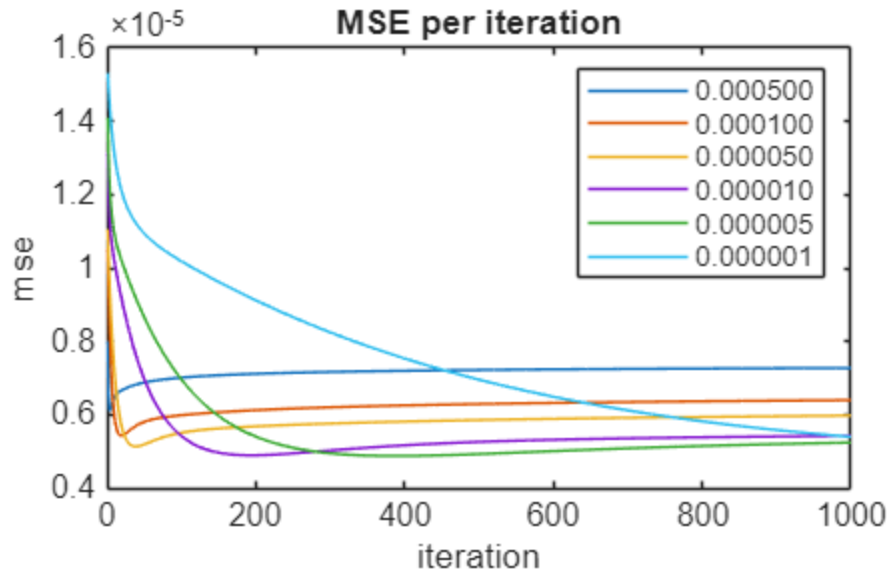


April-21-26

Next steps:

- Are the exciters linear?

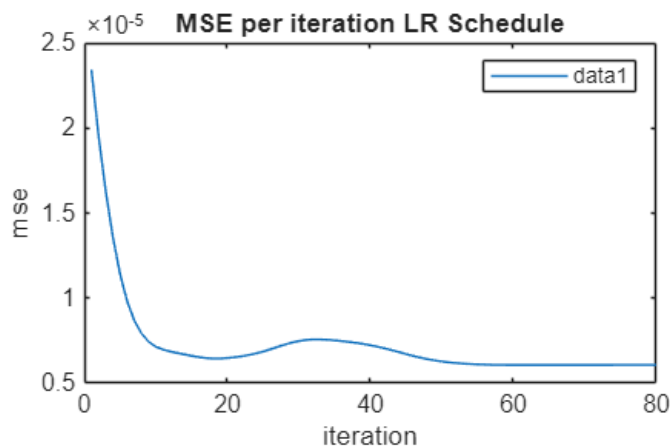
Before testing nonlinearity, I wanted to make sure there was nothing in the software causing issues. I found a small bug where I reset the x buffer and the running dot product for NLMS each iteration, although fixing it didn't do much since the filter is so small compared to the length of the signal. I wanted to see what was going on with the loss curves since they didn't match the simulation data at all. I found that in the current recordings, the amplitude was roughly 10 times larger than the simulation data. I tried using much smaller step sizes to compare the curves



With small step sizes it works better, but you need a lot of iterations which is slow. I tried the following step size schedulers:

- $1 / (2 * t^2)$
- $\max(1e-5, 0.9 * \exp(-t / 5))$
- $\max(1e-5, 1 / (t + 4))$

These worked pretty well, converging to roughly the same value but much faster



$\max(1e-5, 0.9 * \exp(-t / 5))$

None of these, however, got rid of the spiky responses in the data. So I think the issue really is the exciter behavior which I need to look into next.

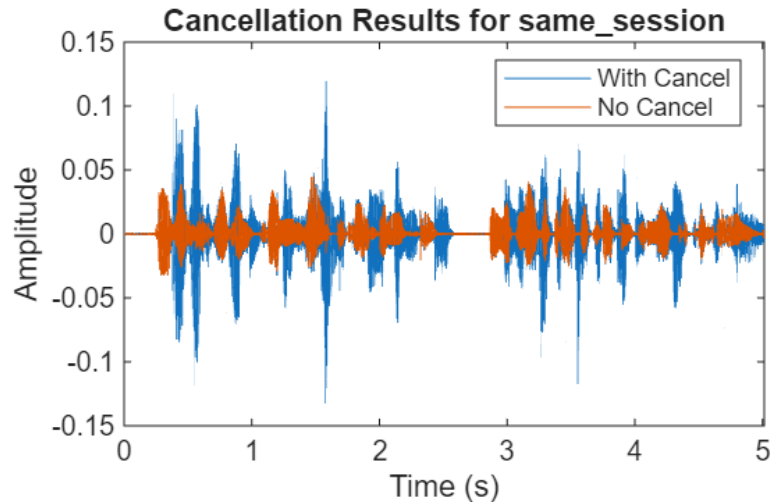
April-15-26

Next steps:

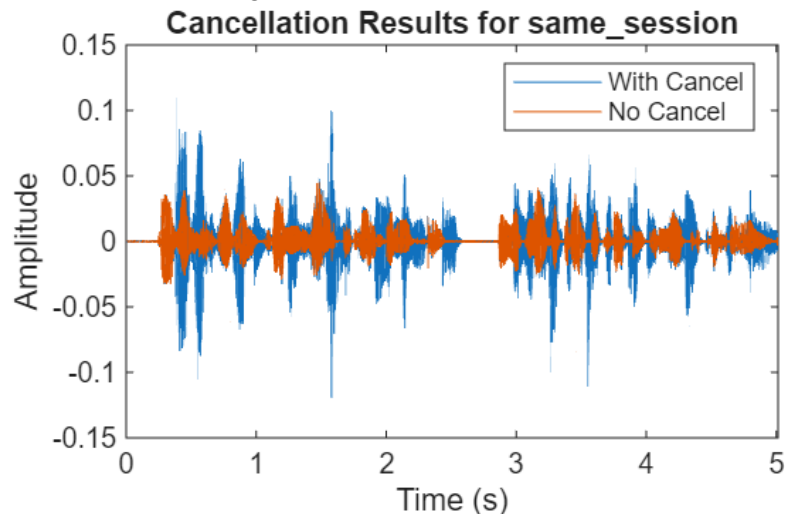
- When using the highpass, f.ref_filt is on the highpassed ref. Is this wrong?
- See if the drivers are behaving nonlinearly
 - Try lower volume?
 - How to test?

The speaker sounded unstable and distorted so I tried moving the exciter to an asymmetric point on the panel. It burnt out so I replaced it with a new one, and the sound quality significantly improved. Apparently it might be easier for this problem to leave it in the center since more modes get cancelled.

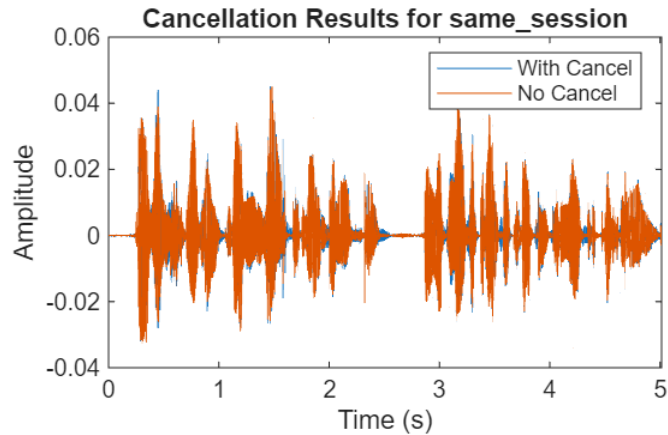
I tried cancellation again without subtracting the direct waves, since the driver is new



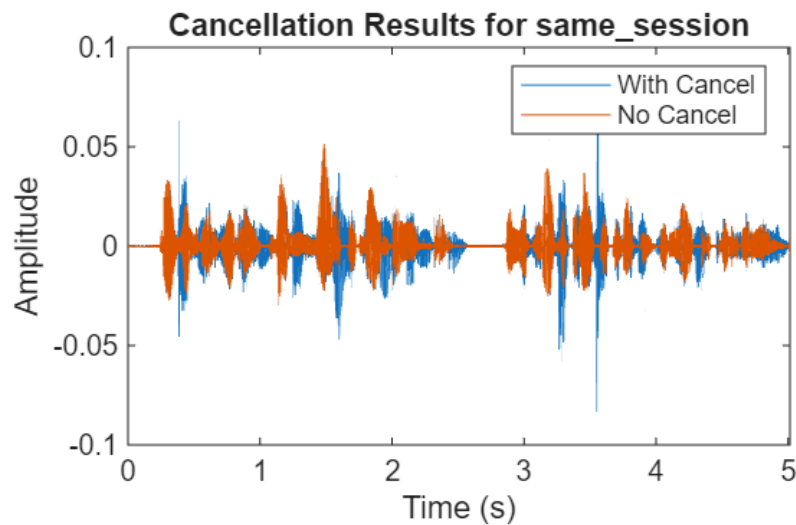
It didn't work. For whatever reason, the filter was struggling to match the target too. I had to use the LMS update instead of NLMS, or the prediction would be spiky and unstable. This meant it didn't match super perfectly, which might have contributed. I then tried subtracting the direct waves from the target to learn the filter.



That didn't help either. For whatever reason, the panel signal sounds audibly time delayed when I play them, and I don't know why. I tried using the prediction from the filter instead of `ref_filt`



It didn't do anything, as expected. I tried replacing the panel with a conventional speaker, remeasuring everything, and then learning a filter.



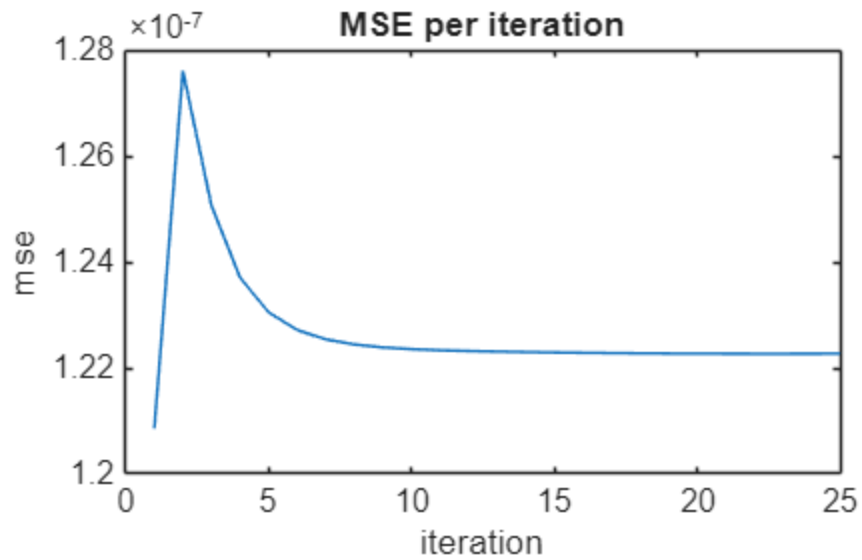
It definitely did better than the panel, but still not working. It didn't sound time delayed though, despite looking like it. It was also harder to learn the filter, I had to play around with it a lot. I don't know what to try next.

April-7-26

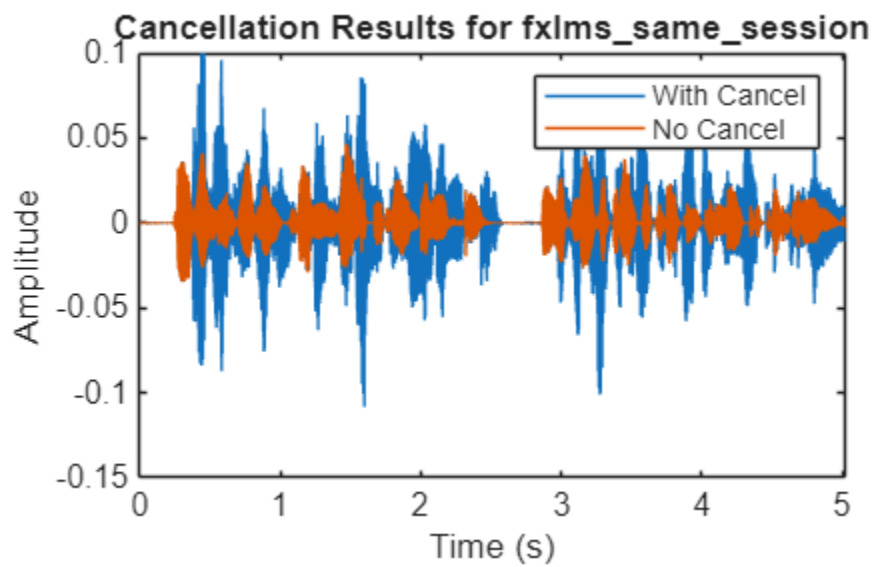
Next steps:

- Try subtracting the direct waves measured with the foam background from the target
- Try using y as the driver instead of ref_filt
- Try using a conventional speaker instead of a panel for ANC

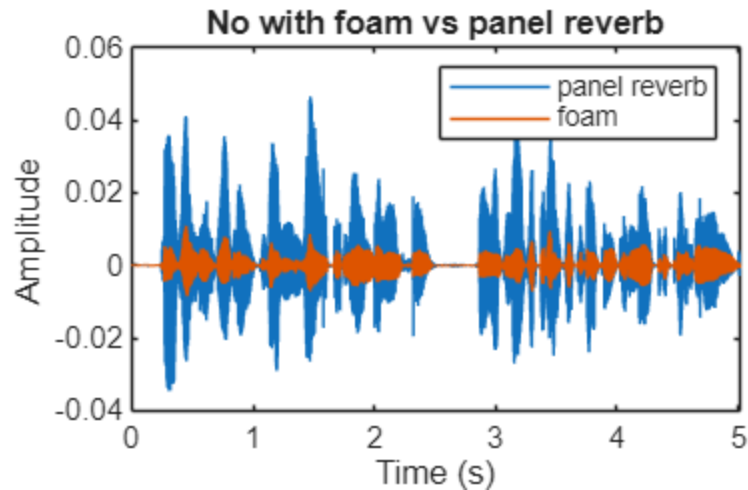
I went back in the lab to attempt a same session cancellation with both the better mics. I used a HP and NLMS update to learn the filter. While we got good MSE, it is still less than I was getting with the premeasured data



I don't know why the curve looks like this here, or why it's an order of magnitude higher than we could get with the premeasured data. Testing it on the speakers, it didn't cancel



My main idea for what could be happening is that perhaps the direct waves are not as negligible as I thought, and the filter is attempting to cancel those too which maybe doesn't work?



I want to try subtracting the direct energy out of the target, learning the filter, and then playing that. I also want to try just playing y , the output of the filter, on the panel. Supposedly you're supposed to do WX , but perhaps WHX the y value can cancel it too. If that doesn't work, I'm gonna try it with a conventional speaker to see if the issue is the panel being complicated.

April-2-26

Next steps:

- Redo the lab measurements when free (Negate the cancellation signal!!!!)

I looked into the effects of normalization and the NLMS update. Since the NLMS update scales the step size by the power of the current window of the signal, when we normalize the signal by dividing out a factor of k , we pick up a k^2 term in NLMS that isn't present in LMS.

LMS update: $w_{n+1} = w_n + \mu x_n e_n$

NLMS update: $w_{n+1} = w_n + \frac{\mu}{\|x_n\|^2} x_n e_n$

F_x-method: use $\tilde{x} = \hat{s} * x$
estimated
IR from plant
to error mic

Add normalization factor k

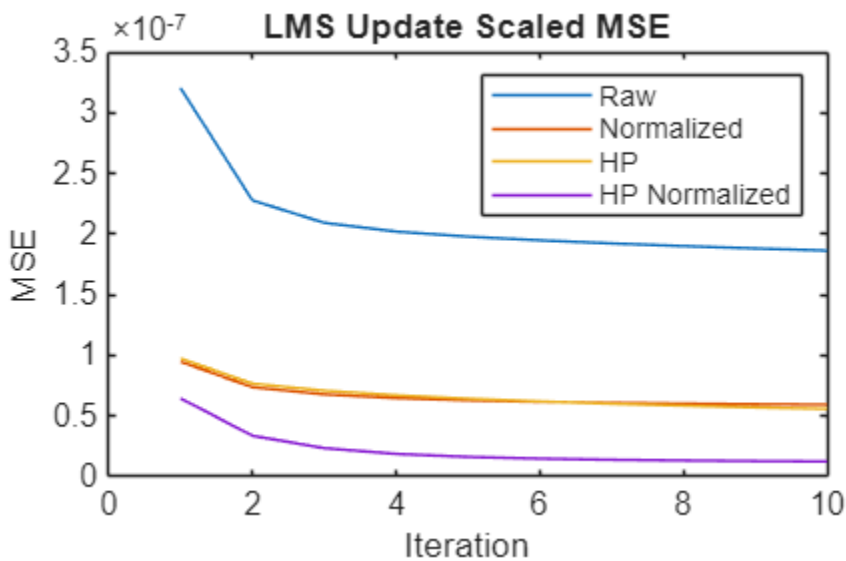
F_x-method: use $k\tilde{x} = \hat{s} * kx$

F_x LMS update: $w_{n+1} = w_n + \mu k x_n e_n$

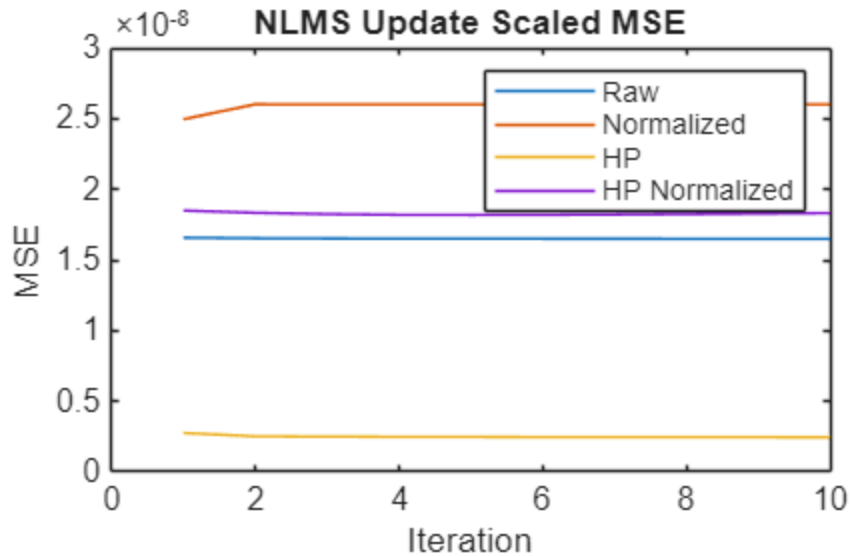
F_x NLMS update: $w_{n+1} = w_n + \frac{\mu}{\|kx_n\|^2} kx_n e_n$
 $= w_n + \frac{1}{k} \frac{\mu}{\|x_n\|^2} x_n e_n$

Made with Goodnotes

I compared Highpass, normalized data, both, and neither, for both LMS and NLMS. I scaled the normalized MSE by a factor of k^2 to compare against the unnormalized data



For the LMS update, normalization always improves over the unnormalized counterpart.



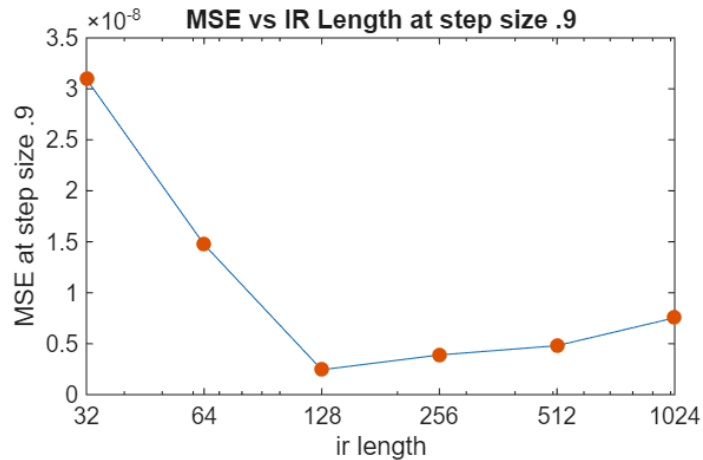
For the NLMS update, the normalized data always does worse than its unnormalized counterpart. This seems to support the idea that the step size gets messed up for normalizing the data with the NLMS update. Specifically, the step size gets really small since k is about .13. It's worth noting that the NLMS update always does better than the equivalent LMS update though. So it seems the optimal choice is a HP filter with NLMS update.

March-31-26

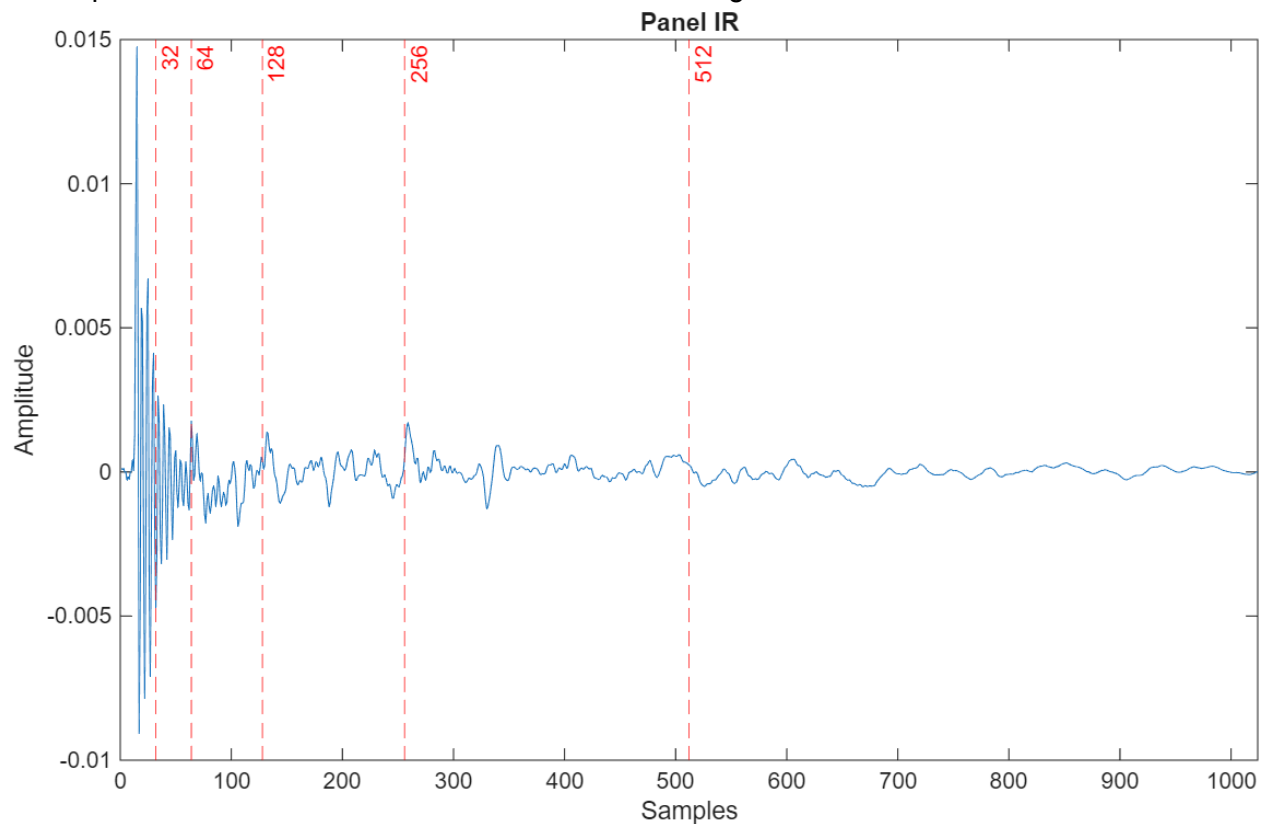
Next steps:

- Why does normalization not help?
- Redo the lab measurements when free (Negate the cancellation signal!!!!!!)

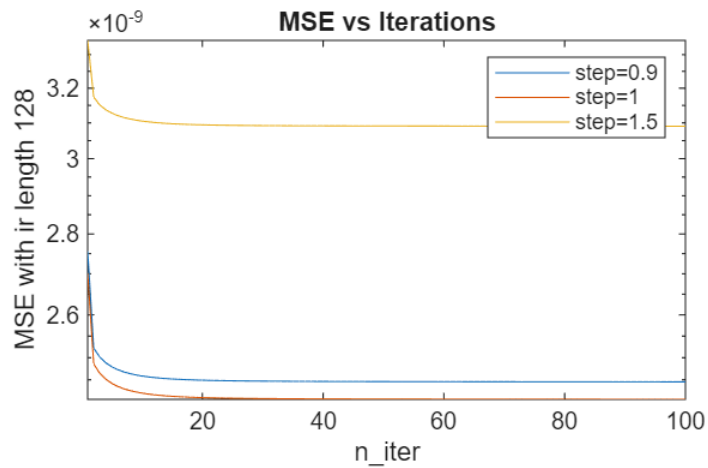
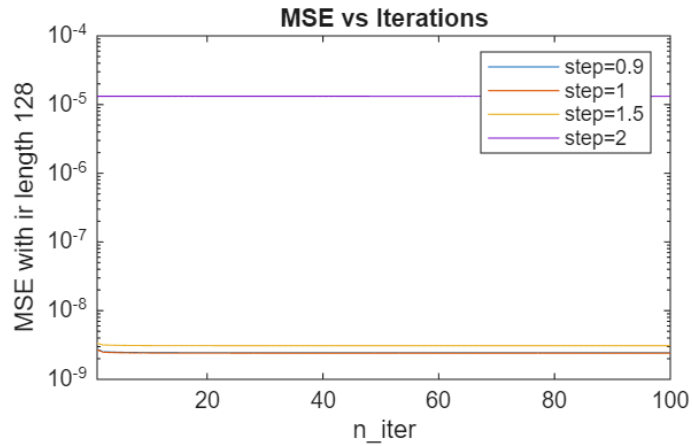
Aaron is still set up in the lab, so I wasn't able to go there. I started by examining the FxLMS code, and it seems to be implemented properly. I did notice that I wasn't negating the predicted reverb signal for cancellation, so I need to do that going forward. I ran a bunch of grid searches over the highpassed data. First, I ran the step_size search over a range of IR lengths, from 32 to 1024.



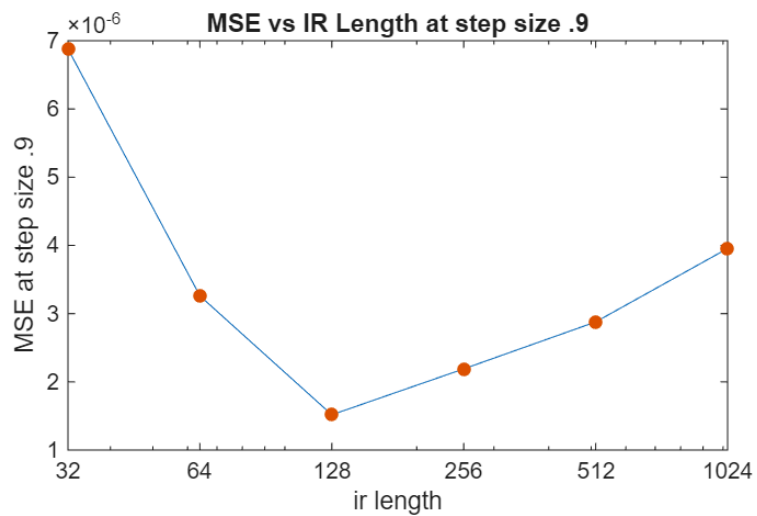
So this shows that smaller doesn't always mean better, but 128 just seems to be the optimal size. I plotted the IR with vertical lines at each of the lengths

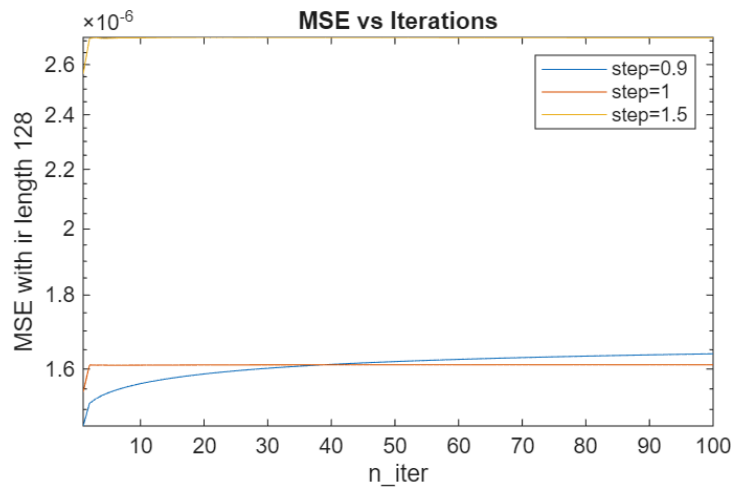
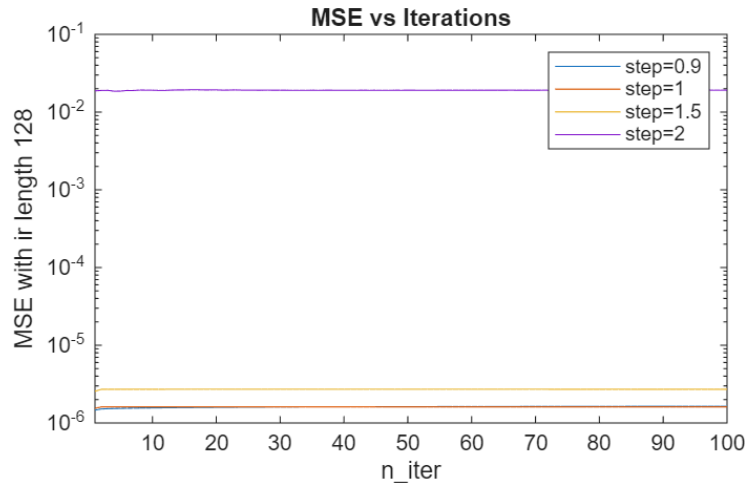


It looks like noise is present after 64, but perhaps the signal is still trying to decay after that. It seems apparent that anything after 128 is probably just additional noise that is throwing off the filter though. The optimal value might be between 64 and 128. I also wanted to see what happened if we increased the step size from .9, so I fixed the IR length to see

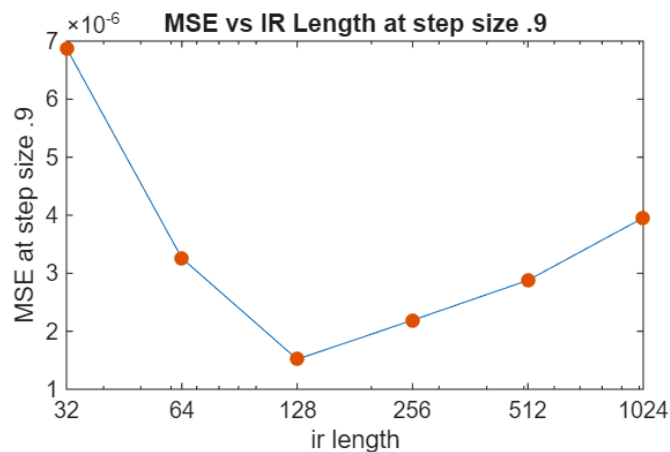


Step size of 1 actually did a bit better than .9, but it started doing worse as the step size increased further. I also tried running all the same tests on normalized data with no highpass, normalizing the ref, target, and IR

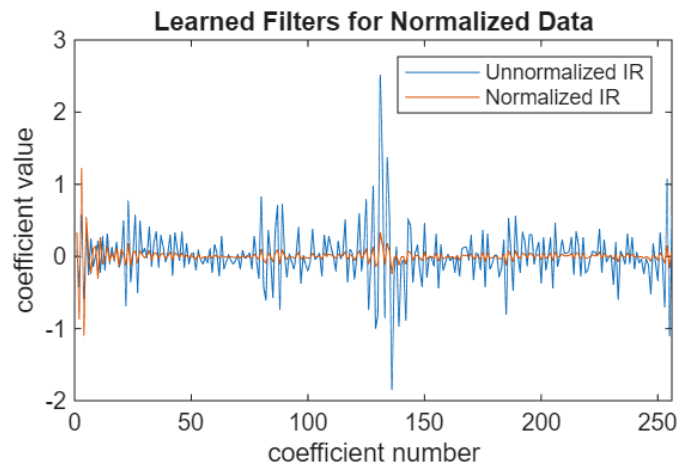




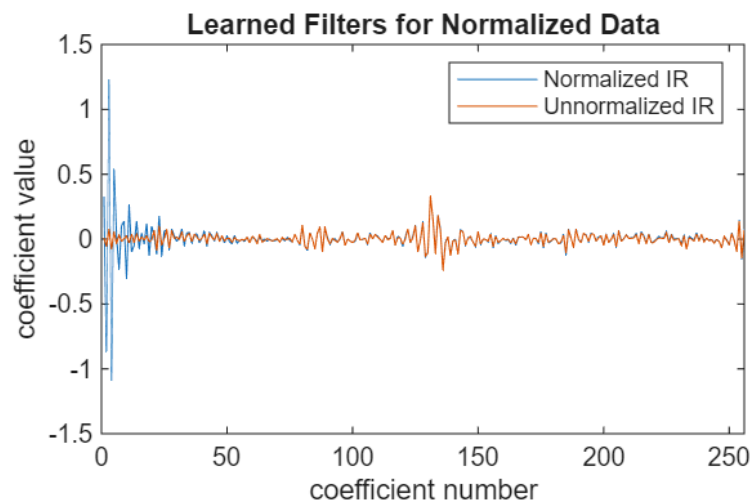
Similar story to the unnormalized data, although again the loss is orders of magnitude higher than when we weren't normalizing. I realized though that if we normalize the panel IR and the ref, then for normalization factor k we're trying to learn $\hat{k}\hat{y} = \hat{k}\hat{S} * kx$ for FxLMS, which means we're picking up an extra normalization factor. I reran the grid search normalizing only the ref and target this time, leaving the panel_ir the same



It's literally the same plot as before, down to 4 decimal places (e-6). That makes no sense. I compared the filters for IR_length 128 and step_size .9



So the filters are different. But they follow kind of similar shapes. If they're the same, then the filter for the normalized IR should be the unnormalized IR * k, so I tried that



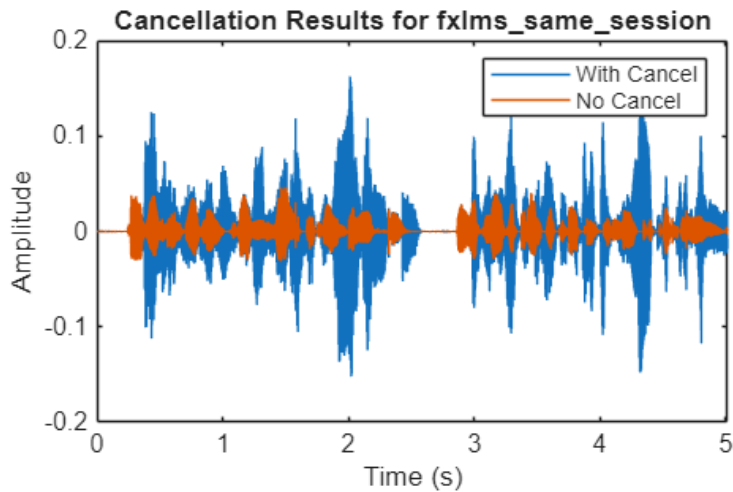
And here it's clear that they are very similar, save for a few artifacts at the end.

March-25-26

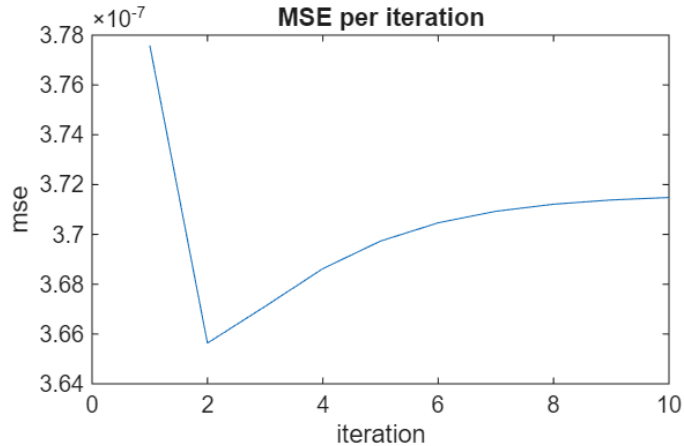
Next steps:

- Why is bigger step size always better?
- Why is smaller IR length usually better?
- Why does normalization not help?
- Redo the lab measurements when free

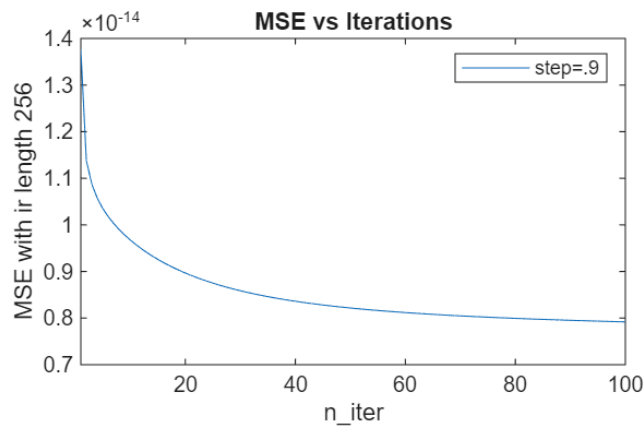
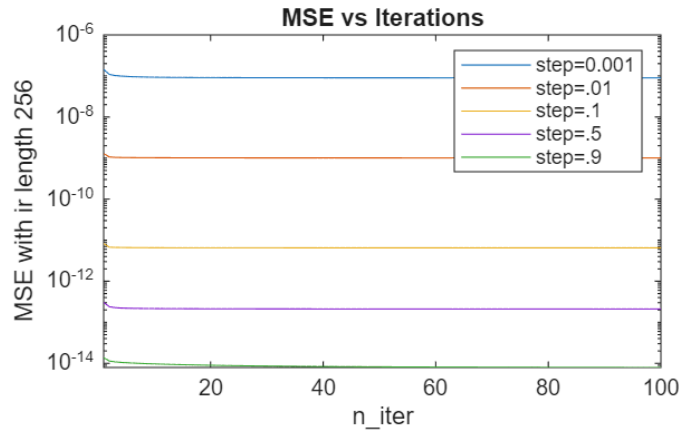
I tried measuring the IRs and reference, learning the filter, and running the cancellation all with the same lab setup. Aaron was set up in the lab so I had to use the lapel mic at the source, and it seemed to be very noisy. I had to play with the gain to make it tolerable, but then the gains were different. The cancellation didn't work, but I think the lapel mic contributed to that



I noticed that the MSE curve didn't seem to improve at the higher step sizes, but it was always better by a significant factor than lower step sizes



I tried running a grid search over the step sizes to investigate this. It seems like it is always true, and with the cleaner data I've been training on, the lower step sizes actually go down



And for some reason, now we're hitting 10^{-14} error which we've never seen before. I don't really understand what's going on, everything seems to be backward.

March-18-26

Next steps:

- Is the issue that we're learning with old IRs?
- Why does normalization not help?
- Does the MSE decrease with iterations always for longer IRs?
- Can we get better results with more iterations?

I ran the FxRLS and FD-FxLMS in the lab, and they both slightly increased the energy picked up by the mic. I'm not sure why nothing is able to cancel any reverb. At some point I want to try re-measuring the IR, learning the filter, and testing it all in the same session to ensure setup issues aren't the root cause. I also want to figure out why normalization makes it worse, that seems to make no sense.

March-17-26

Next steps:

- Does the MSE decrease with iterations always for longer IRs?
- Can we get better results with more iterations?
- How does 1 iteration of FxRLS do in the lab?
- How does FD-FxLMS do in the lab?
- Is the issue that we're learning with old IRs?
- Why does normalization not help?

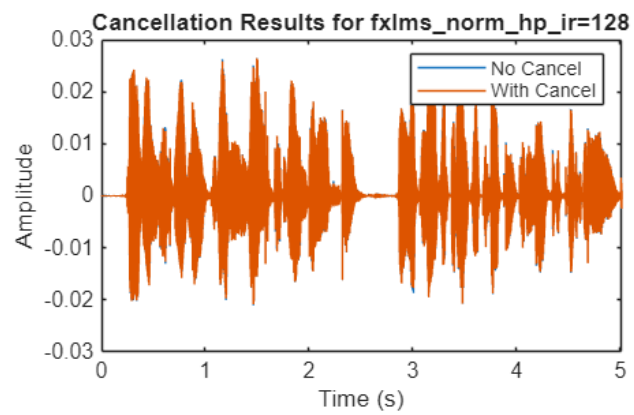
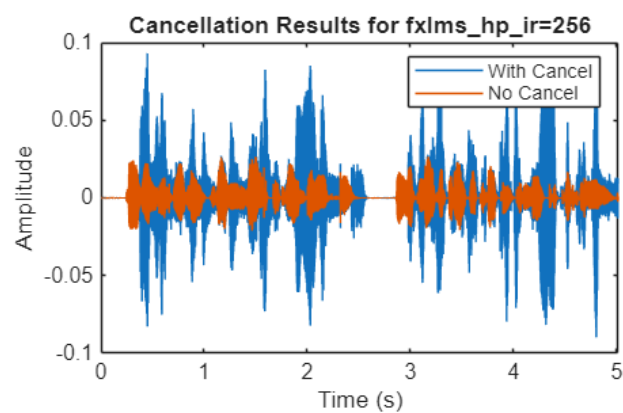
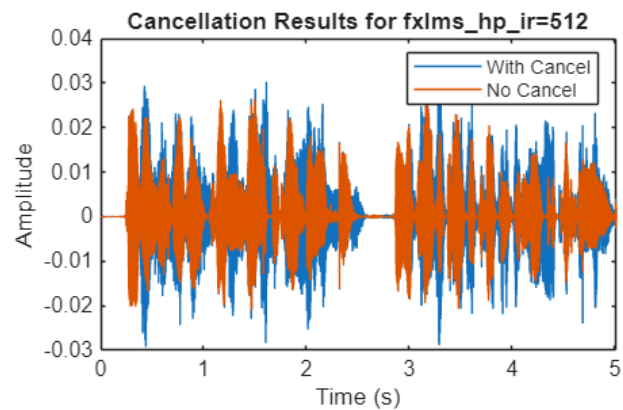
I ran the grid search on FxRLS and FD-FxLMS, and tried combos of normalizing and adding the highpass. For both of them, just the highpass did better than when we added normalization, which matches the FxLMS case. I don't really know why this should be the case. I still need to test them in the lab to see if they cancel better than the FxLMS attempts.

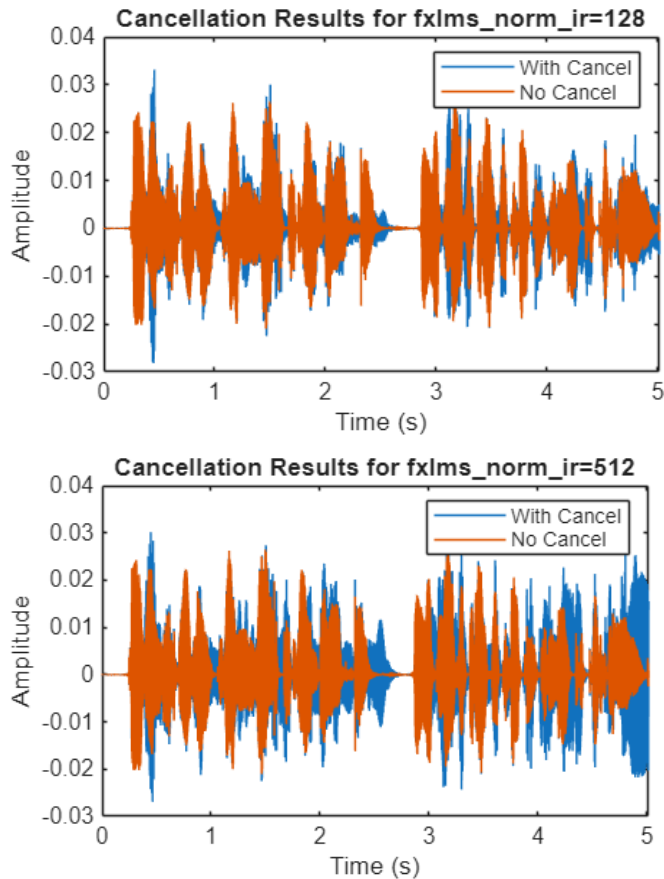
March-03-26

Next steps:

- Does the MSE decrease with iterations always for longer IRs?
- Can we get better results with more iterations?
- How does 1 iteration of FxRLS do in the lab?
- How does FD-FxLMS do in the lab?
- Is the issue that we're learning with old IRs?
- Is it struggling because simulink would play the clip twice?

I tested highpass, normalization and highpass, and normalization with no highpass in the lab for all four IR lengths to compare. None of them were able to make an improvement. It seemed like normalization with the highpass did absolutely nothing. Just the highpass was very inconsistent, with some IR lengths not doing too much worse than no cancellation, and some of them doing much worse. Just normalization didn't do much worse than without cancellation, although there were some weird artifacts.





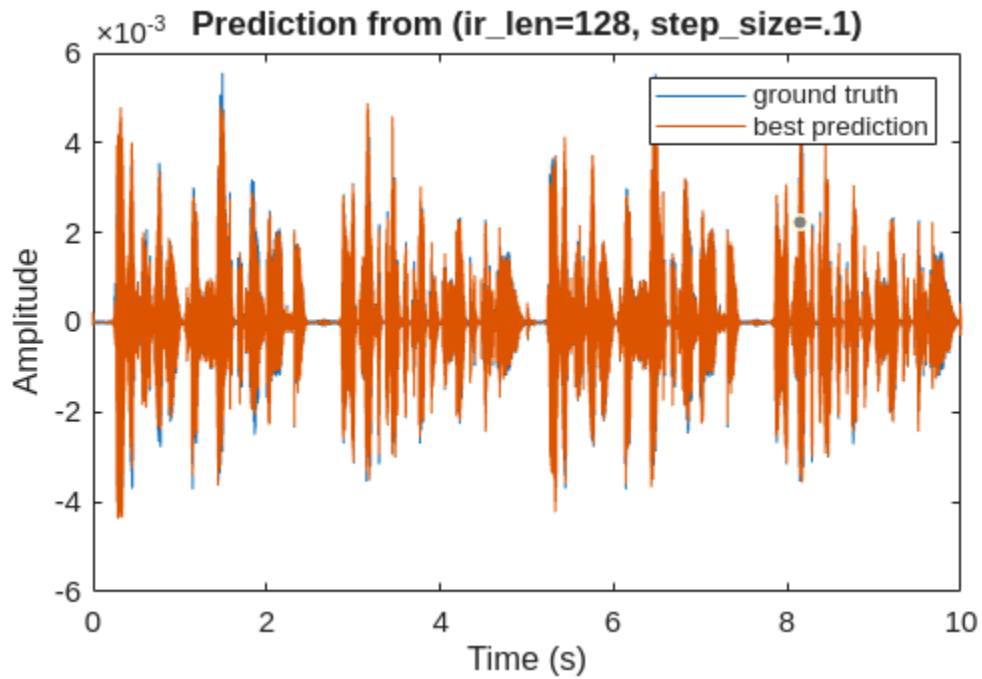
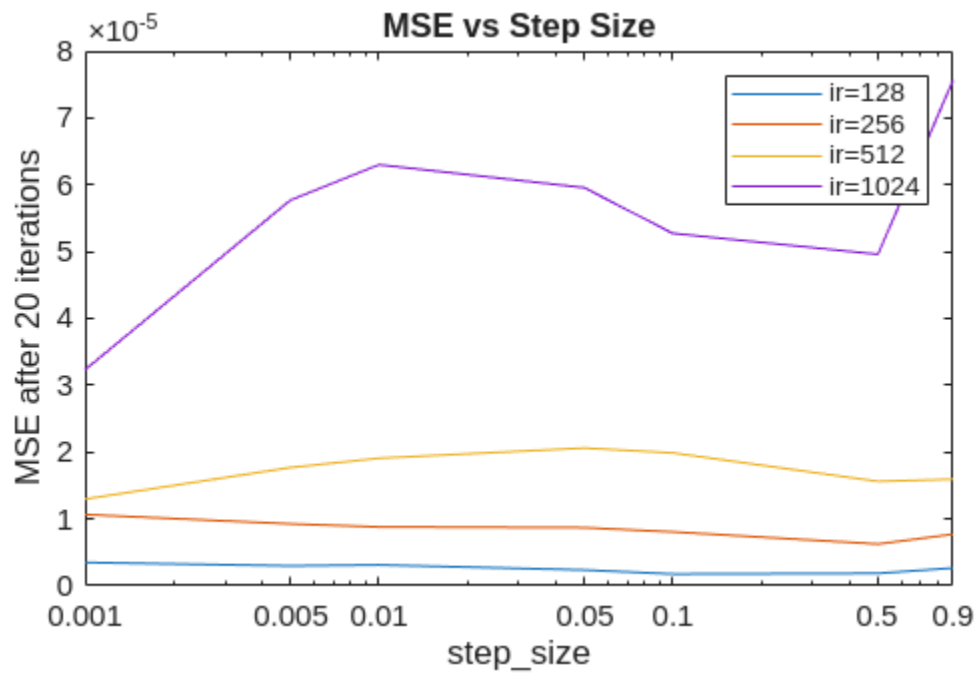
March-02-26

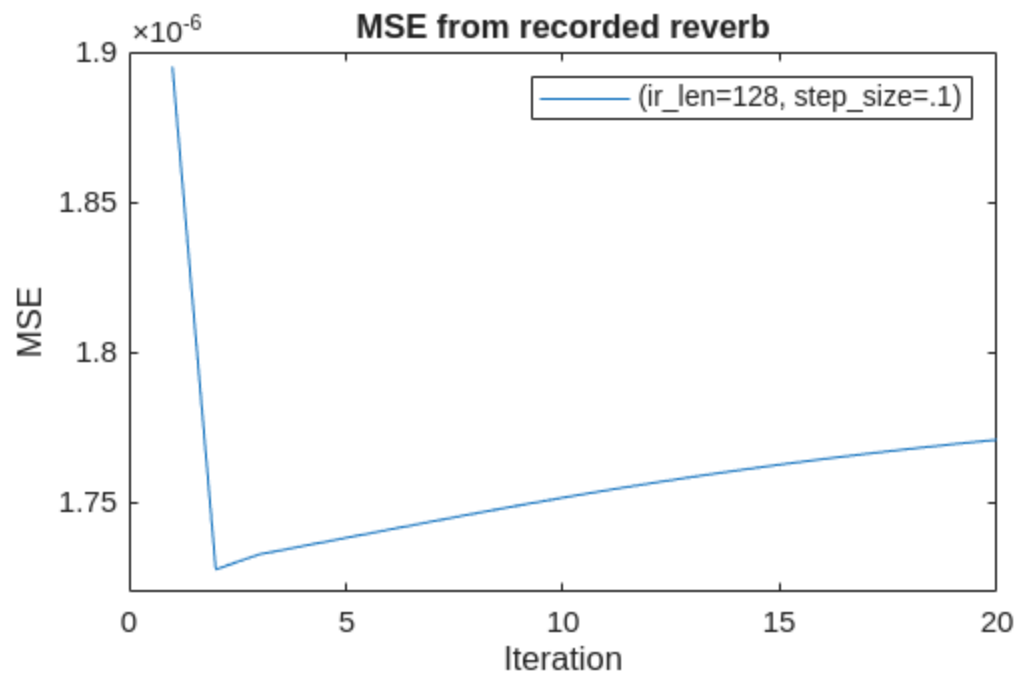
Next steps:

- Test current results in the lab and compare

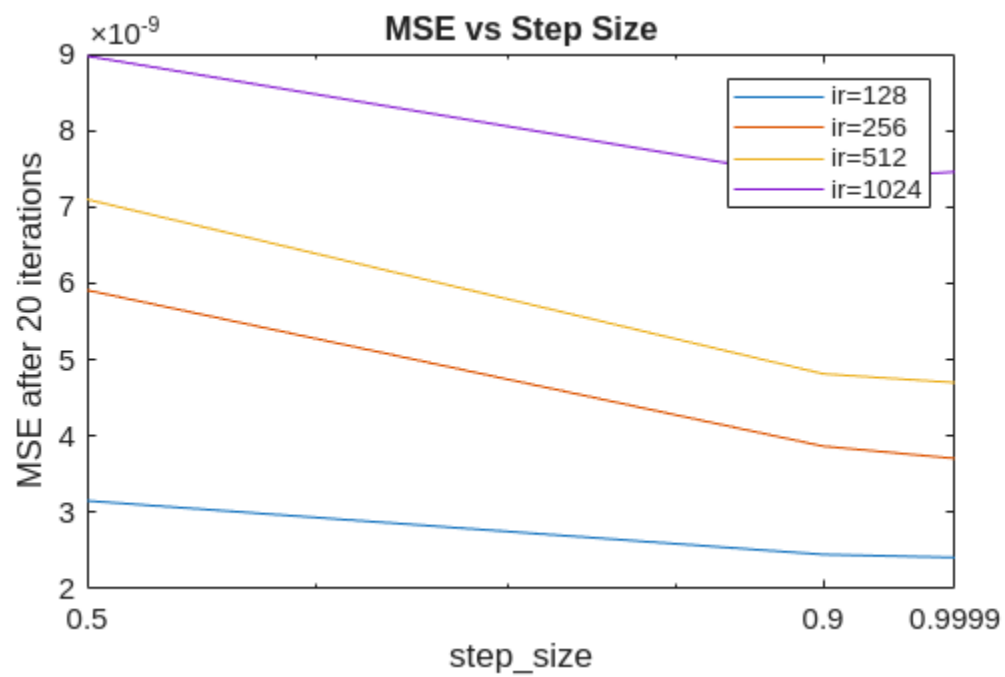
I tried increasing the step size beyond 1, but there were NaNs, probably from exploding values. So I had to normalize instead. It seems like even considering different combinations of normalization and highpass filtering, the smallest IR length performs the best. However, they all seem to get pretty good matches. The MSE doesn't always go down though, which is weird, I wonder if the smaller filters are overfitting. I want to test highpass, norm and highpass, and just norm in the lab for different IR lengths to compare them all.

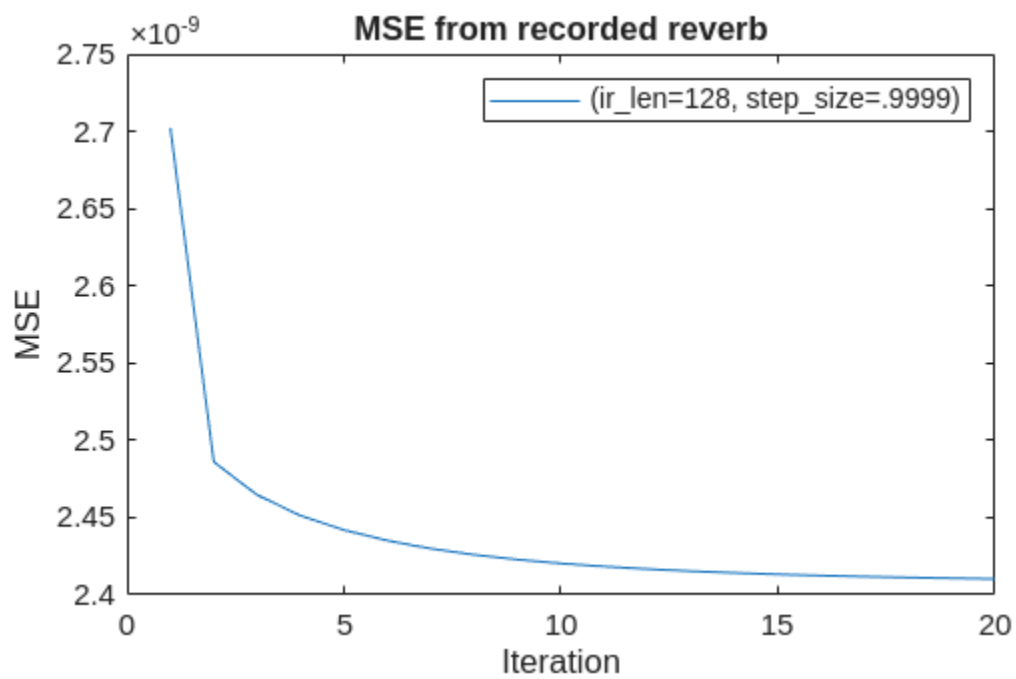
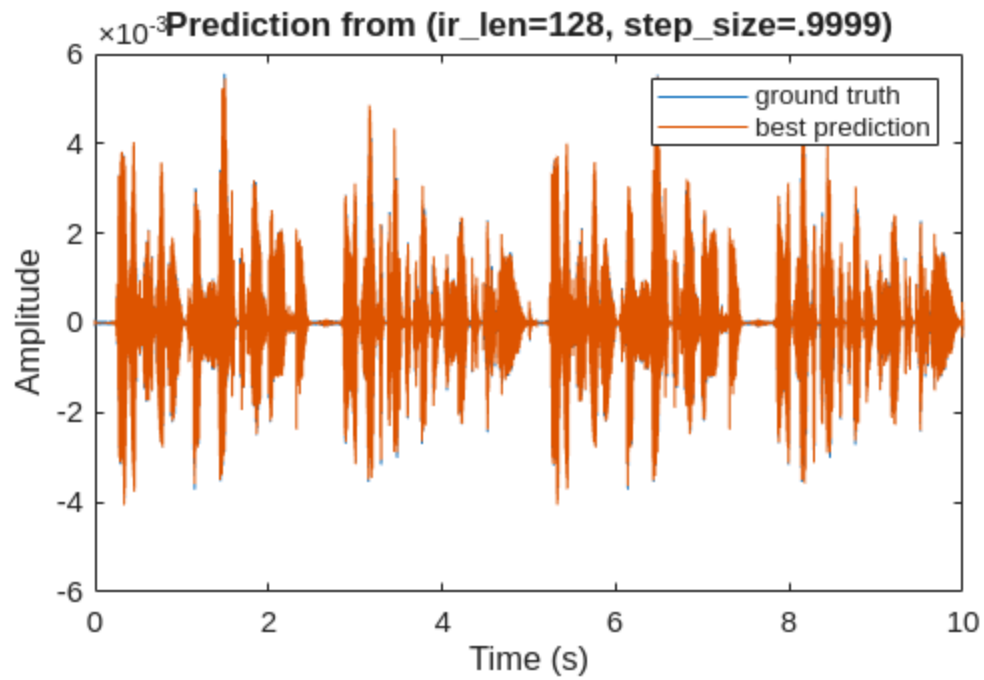
Norm and Highpass



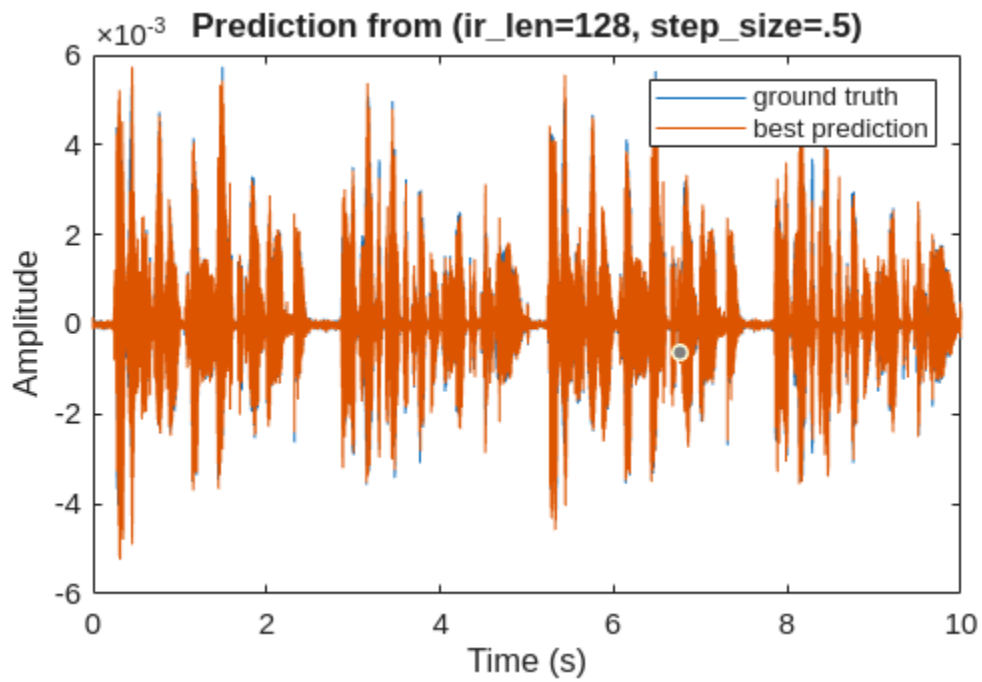
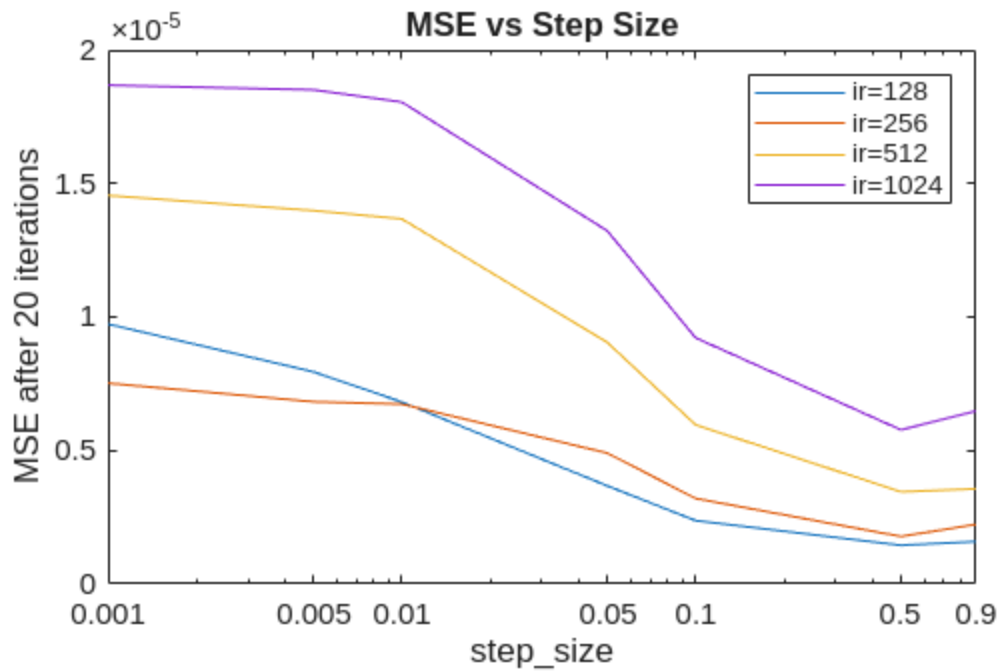


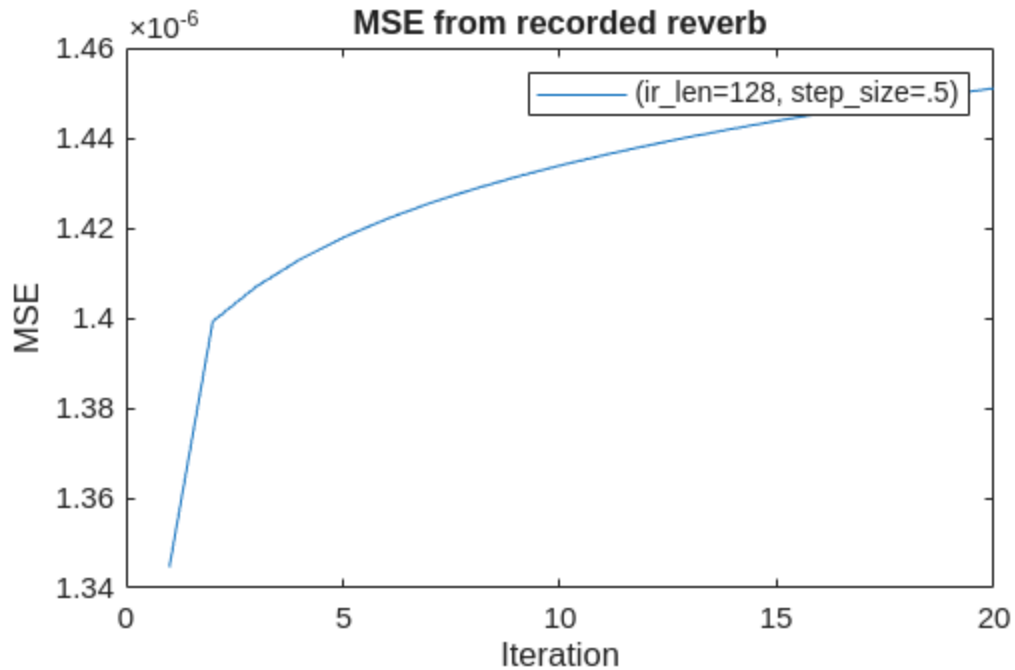
Just highpass





Just Normalized



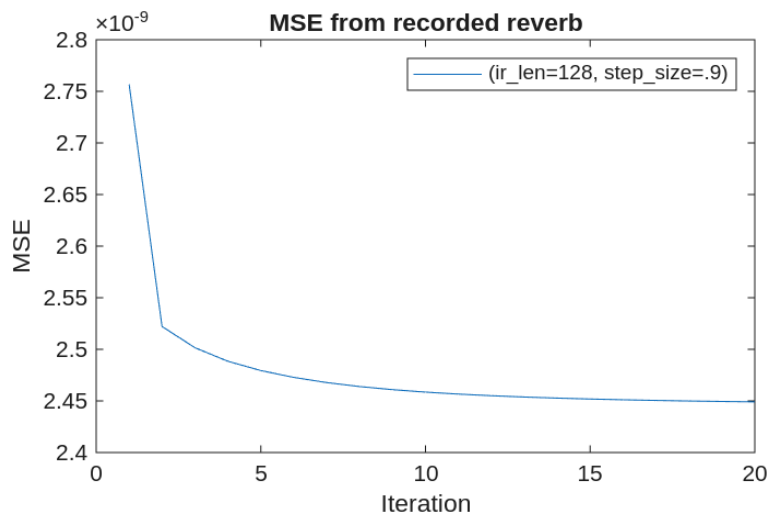
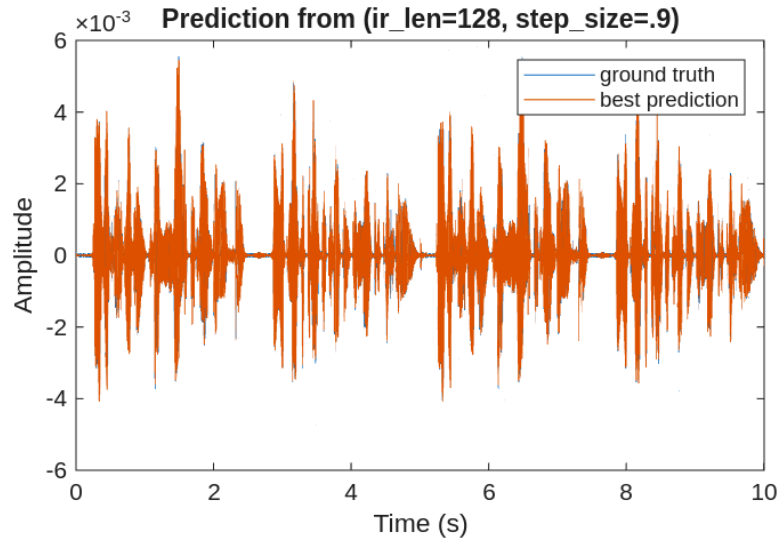


Feb-26-26

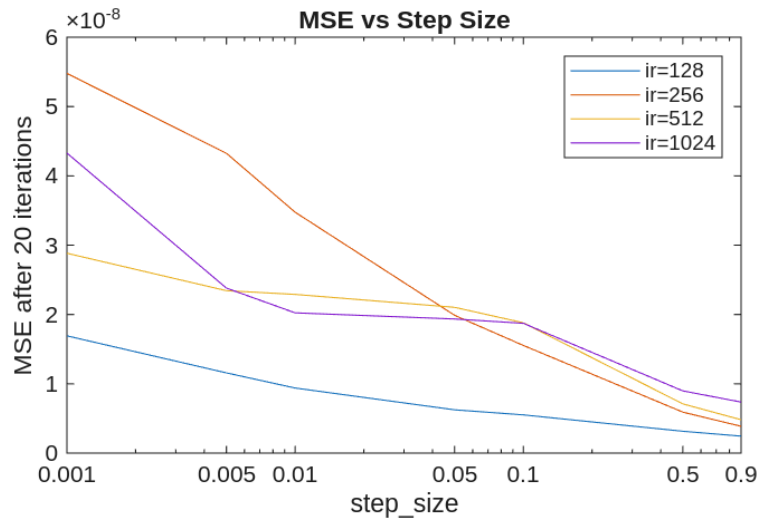
Next steps:

- Normalize the data and grid search from there
- More iterations for longer filters?
- Compare the lab results for different filter lengths

The results from the grid search on FxLMS params showed the best MSE from ir_len=128 and step_size=0.9.



This is very interesting because we would have expected the longer filters to be able to learn more. Additionally, the step size seems kind of large. But perhaps this is because the data is unnormalized; since the data are on the order of $e-3$, perhaps we need to scale the step size up by 100-1000. This might also be affecting the ir_len, because a smaller step size makes it harder to learn bigger filters, or at least it makes it take longer.



I tested this filter in the echo room, and it seems to not work very well. It doesn't sound inappropriately delayed, but it's louder than I would've expected. I wonder if this is perhaps a result of using a small filter that isn't capturing the full IR properly. I need to play around with lowering the MSE for larger filters and then compare them all to see the difference in the lab.

